```
[DeadLetterActorRef] - Message [ask.simple.Worker$Done$]
➥to Actor[akka://example-ask-without-content/deadLetters]
➥was not delivered. [1] dead letters encountered. If this is
➥not an expected behavior then
➥Actor[akka://example-ask-without-content/deadLetters] may have
➥terminated unexpectedly. This logging can be turned off or
➥adjusted with configuration settings 'akka.log-dead-letters'
➥and 'akka.log-dead-letters-during-shutdown'.
```

❶ Texts processed on time

❷ Late answer

If you look at the error in parsing `text-c`, you will see that three related things happened:

- The manager no longer waited for the response. It registered a failure and reported that the task was not completed:

  ```
  [Manager$] - parsing 'text-c' has failed with [Ask timed out ...
  ```

- On the worker side, the worker completed the task and logged the fact that it completed the parsing. That's why you see the following message even though the parsing took longer than the manager was willing to wait:

  ```
  [Worker$] - worker-text-c: DONE!
  ```

- The third point concerns the `deadLetters` actor mentioned in chapter 2. When the worker finished its task, it tried to send a response, but this message never reached the manager. This happened because asking goes through an intermediate actor, like the adapter from the previous example.

  An ephemeral actor is created that acts as a proxy between the manager and the worker responsible for the timeout. When the time is up, the proxy actor sends an error message to the manager and stops. Because the manager has stopped, the